

MODULE 2 - OUTILS QUANTITATIFS

LOGICIEL R 1

Cours et travaux dirigés

Julie Scholler



TABLE DES MATIÈRES

I	NOTES DE COURS	2
	PRÉSENTATION DE L'ENSEIGNEMENT	3
	THÈME 1. PRÉSENTATION GÉNÉRALE DU LOGICIEL R ET DE RSTUDIO	4
	THÈME 2. PREMIÈRES MANIPULATIONS	6
	THÈME 3. IMPORTATION ET EXPORTATION DE DONNÉES	14
	THÈME 4. EXPORTATION DE GRAPHIQUES	19
II	TRAVAUX PRATIQUES	20
	PRÉSENTATION DE L'ENSEIGNEMENT	21
	TP 1. PRÉSENTATION GÉNÉRALE DU LOGICIEL R	22
	TP 2. DATA FRAMES ET STATISTIQUES UNIVARIÉES QUANTITATIVES	27
	TP 3. CALCUL MATRICIEL ET TABLEAUX DE CONTINGENCE	32
	TP 4. IMPORTATION DE DONNÉES ET VARIABLES QUALITATIVES	36
	TP 5. MANIPULATIONS AVANCÉES DE DATA FRAME	39
	EXERCICES D'ANNALES	41

PREMIÈRE PARTIE

NOTES DE COURS

PRÉSENTATION DE L'ENSEIGNEMENT

INTERVENANTS

Franck Pillier : franck.pillier@univ-tours.fr, bureau B246 (bâtiment B)

Julie Scholler : julie.scholler@univ-tours.fr, bureau B246 (bâtiment B)

OBJECTIFS

Le but de cet enseignement est de se familiariser avec le logiciel R et d'apprendre à utiliser les outils de gestion de données ainsi que les outils statistiques de base du logiciel R .

CONTENU DES TRAVAUX PRATIQUES

- Découverte du logiciel R : interface RStudio , manipulations et objets de base
- Études descriptives de données enregistrées dans un *data frame*
- Représentations graphiques
- Importation, exportation et manipulation de données

APPROCHE ET SUPPORTS PÉDAGOGIQUES

L'enseignement est composé de 2 séances d'une heure de cours magistral et de 5 séances de travaux pratiques de 2h. Il n'y a pas de séances toutes les semaines, surveillez votre emploi du temps.

Les sujets de TP, ainsi que des corrections partielles, sont disponibles sur mon site perso (<https://juliescholler.gitlab.io>) et via l'ENT sur la page Céline du cours.

MODALITÉS D'ÉVALUATION

- **Session 1** : contrôle continu
- **Session 2** : examen sous forme d'un exercice sur ordinateur

La note de contrôle continu prendra en compte l'évaluation d'exercices à effectuer en dehors des heures de TP et d'une épreuve sur poste informatique en fin de semestre.

PRÉSENCE

La présence en TP est obligatoire. En cas d'absence, vous devez présenter un justificatif ou une justification au chargé de TP dans les 8 jours.

BIBLIOGRAPHIE

Il s'agit de lectures complémentaires aux travaux pratiques. On trouve également de nombreuses ressources sur internet.

- *Statistiques avec R* , Cornillon Pierre-André et Autres (519.5 STA)
- *Le logiciel R* , Lafaye de Micheaux Pierre et Autres (519.5 LAF)
- *Comprendre et réaliser les tests statistiques à l'aide de R* , Millot (519.5 MIL)
- *Initiation à la statistique avec R* , Frédéric Bertrand et Myriam Maumy-Bertrand (519.5 BER)

THÈME 1. PRÉSENTATION GÉNÉRALE DU LOGICIEL R ET DE RStudio

PRÉSENTATION DE R

R est un langage orienté vers le traitement de données et l'analyse statistique. Il s'agit également d'un logiciel libre publié sous licence GNU GPL.

Il permet de réaliser des analyses statistiques telles que

- des statistiques descriptives : moyenne, médiane, variance, etc.
- des tests d'hypothèses et des intervalles de confiance
- des régressions linéaires
- de l'analyse factorielle
- du machine learning
- des graphiques

AVANTAGES

- multiplateforme (Linux, Mac oS X, Windows)
- gratuit
- très puissant car les fonctionnalités de base peuvent être étendues à l'aide d'extensions (plus de 10 000) :
- possibilités de manipulation de données supérieures à un tableur
- bonnes capacités graphiques et nombreuses possibilités d'export
- les méthodes statistiques récentes sont rapidement disponibles
- communauté d'utilisateurs et de développeurs très active et réactive
- beaucoup d'aide, d'informations et de forum à ce propos sur le web

INCONVÉNIENTS

- logiciel et documentation de base en anglais (mais de plus en plus de ressources en ligne en français)
- R s'apparente davantage à un langage de programmation qu'à un logiciel proprement dit

INSTALLATION DE R

Sur le site <http://www.r-project.org/>, effectuer la démarche suivante :

- rubrique **Download**, cliquer sur **CRAN**
- choisir un site miroir en France
- choisir la version en fonction de votre système d'exploitation (pour Linux, il y a de fortes chances que R soit directement disponible via le gestionnaire de paquets).

INSTALLATION DE **RStudio**

Sur le site <https://www.rstudio.com/>, effectuer la démarche suivante :

- dans la catégorie **Products** choisir **RStudio the premier IDE for R**
 - sur la nouvelle page, scroller jusqu'au bouton **Download RStudio Desktop**
 - sur la nouvelle page, scroller jusqu'à pouvoir choisir la version en fonction de votre système d'exploitation (pour Linux, il y a des chances que RStudio soit directement disponible via le gestionnaire de paquets).
-

THÈME 2. PREMIÈRES MANIPULATIONS

PRINCIPES GÉNÉRAUX DU LOGICIEL R

La fenêtre de commandes (console) du logiciel R permet d'exécuter des instructions (commandes ou expressions).

R EST UNE CALCULATRICE

R permet de faire les opérations de calcul élémentaire.

Exemples :

```
3 + 5 #addition
## [1] 8
2 * 4 #multiplication
## [1] 8
#3,5-8 # problème avec la virgule
3.5 - 8 # le bon séparateur décimal est le point
## [1] -4.5
```

```
2.3 + 9 - 5.1
## [1] 6.2
3*2 - 5*(2-4)/6.03
## [1] 7.658375
3^2 # puissance
## [1] 9
```

R possède en mémoire la valeur de quelques constantes mathématiques, comme la constante π qui est appelée par la commande `pi`.

```
pi
## [1] 3.141593
```

R permet de faire des calculs plus élaborés. Il utilise pour cela des fonctions.

Exemples :

```
sqrt(2) # racine carrée
## [1] 1.414214
log(2) # logarithme népérien
## [1] 0.6931472
```

```
round(pi, 2) # pi arrondi à 2 chiffres après la virgule
## [1] 3.14
```

CRÉATION D'OBJETS

On peut stocker en mémoire des données, des résultats, etc. Pour cela, on définit des objets R (on reviendra sur les différents types d'objets), à l'aide du symbole `<-` qui permet d'assigner une valeur à un objet.

Exemples :

```
a <- 5
a
## [1] 5

b<- a + 1
b
## [1] 6
```

REMARQUE : pour nommer un objet, on peut utiliser un ou plusieurs caractères alphanumériques et éventuellement des points. Cependant il faut toujours commencer par une lettre.

Certains objets existent déjà dans R. Exemple :

```
state.area
## [1] 51609 589757 113909 53104 158693 104247 5009 2057 58560 58876
## [11] 6450 83557 56400 36291 56290 82264 40395 48523 33215 10577
## [21] 8257 58216 84068 47716 69686 147138 77227 110540 9304 7836
## [31] 121666 49576 52586 70665 41222 69919 96981 45333 1214 31055
## [41] 77047 42244 267339 84916 9609 40815 68192 24181 56154 97914
```

Pour comprendre ces données, on peut faire appel à l'aide fournie dans R via la commande `help(state.area)`.

SCRIPT

Afin de sauvegarder son travail ou de faire des rapports, il est utile de créer des scripts. Il s'agit d'un fichier texte contenant une succession de commandes R.

Dans l'onglet **File** de RStudio, choisir **New File** puis **R Script**. Enregistrer votre fichier à l'emplacement de votre choix.

Pour tout travail, je vous conseille de taper directement vos commandes dans le script, puis de les exécuter dans la console en utilisant la fonction **Run**.

Il est possible d'insérer des commentaires dans vos scripts en les faisant précéder du caractère `#`.

```
# ceci est un commentaire.
```

DONNÉES DANS R

R permet de manipuler des données organisées en structures de différentes formes (vecteurs, tableaux, etc.). Toutes ces structures sont composées d'éléments de base, ces derniers pouvant être de différents types (numériques, caractères, etc.).

MODE D'UN OBJET

Les principaux modes d'un objet de R sont :

- **numeric** (valeur numérique) : 1, pi, 3.1416 ;
- **logical** (booléen, valeur logique) : TRUE, FALSE, T, F ;
- **character** (chaîne de caractères) : "blabla".

Pour connaître le mode d'un objet `x` de R, il suffit d'exécuter la commande : `mode(x)`.

Il est possible de tester l'appartenance d'un objet à un mode en particulier avec les commandes suivantes.

```
is.logical(a)
## [1] FALSE
```

```
is.numeric(a)
## [1] TRUE
is.character(a)
## [1] FALSE
```

Il est possible de convertir un objet d'un mode à un autre.
Exemple :

```
c <- as.character(a)
c
## [1] "5"
```

```
is.character(c)
## [1] TRUE
```

Il faut être prudent dans les conversions et bien vérifier comment R convertit.

VALEUR MANQUANTE

Pour différentes raisons, il se peut que certaines données ne soient pas récoltées pendant une étude. On parle alors de données ou valeurs manquantes. Elles sont notées **NA**, pour *Not Available*. Ce n'est pas un véritable mode et il possède ses propres règles de calcul (il faudra être vigilant quand on traitera des données).

Pour savoir s'il existe une donnée manquante dans un objet **x**, il faut poser la question `is.na(x)` (`sum(is.na(x))` pour avoir le nombre de données manquantes).

ATTRIBUT D'UN OBJET

Il y a des attributs toujours présents :

- mode : `mode`;
- longueur : `length`.

Il y a des attributs spécifiques qui varient selon le type d'objet. On les obtient à l'aide de la commande `attributes(objet)`.

PREMIÈRE STRUCTURE DE DONNÉES : VECTEURS

C'est un objet composé d'un ensemble de valeurs toutes du même mode (numérique, logique, etc.). Le nombre d'éléments constitue l'attribut longueur.

CONSTRUCTION

Différentes méthodes sont possibles.

- Construction par la fonction collecteur `c()` :

```
v <- c(10, 4, 5, 8, 3, 2.1, 15, 789, 63, -2)
v
## [1] 10.0  4.0  5.0  8.0  3.0  2.1 15.0 789.0 63.0 -2.0
c(v, 12)
## [1] 10.0  4.0  5.0  8.0  3.0  2.1 15.0 789.0 63.0 -2.0 12.0
c(TRUE, TRUE, FALSE)
## [1] TRUE TRUE FALSE
c(1>0, 1==1, T, 1<0.5)
## [1] TRUE TRUE TRUE FALSE
```

- Création par l'opérateur séquence `seq()` :

```
seq(1, 8, by = 0.5)
## [1] 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0 5.5 6.0 6.5 7.0 7.5 8.0
seq(1, 2, length = 5)
## [1] 1.00 1.25 1.50 1.75 2.00
```

- Création par la fonction répétition `rep()` :

```
rep(1, 4)
## [1] 1 1 1 1
rep("A", 10)
## [1] "A" "A" "A" "A" "A" "A" "A" "A" "A" "A"
```

- encore une :

```
1:5
## [1] 1 2 3 4 5
```

CALCULS

Voici quelques exemples de manipulation de vecteurs.

```
v0 <- 1:5
v0 > 4
## [1] FALSE FALSE FALSE FALSE TRUE
sqrt(v0)
## [1] 1.000000 1.414214 1.732051 2.000000 2.236068
```

```
sum(v0)
## [1] 15
sum(v0 >= 4)
## [1] 2
cumsum(v0)
## [1] 1 3 6 10 15
```

```
v1 <- c(-3, 1.2, NA, 5, NA)
mode(v1)
## [1] "numeric"
is.na(v1)
## [1] FALSE FALSE TRUE FALSE TRUE
```

```
sum(is.na(v1))
## [1] 2
sum(v1)
## [1] NA
v1^2
## [1] 9.00 1.44 NA 25.00 NA
v2 <- c(10, 8, 11, 7, 12)
```

```
sort(v2)
## [1] 7 8 10 11 12
v0 + v2
## [1] 11 10 14 11 17
v1 + v2
## [1] 7.0 9.2 NA 12.0 NA
v0 * v2
## [1] 10 16 33 28 60
```

SÉLECTION D'UNE PARTIE D'UN VECTEUR

On peut sélectionner une partie des éléments d'un vecteur en spécifiant les indices des termes nous intéressant. Voici quelques exemples de sélection d'une partie d'un vecteur.

```
v3 <- seq(0.1, 1, 0.1)
v3[6] # 6ème élément du vecteur
## [1] 0.6
v3[6:8] # les éléments du vecteur des positions 6 à 8
## [1] 0.6 0.7 0.8
v3[c(1, 8, 3, 1)]
## [1] 0.1 0.8 0.3 0.1
v3[-2] # tous les éléments de v3 sauf le deuxième
## [1] 0.1 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0
v3[-c(2, 3, 5:8)]
## [1] 0.1 0.4 0.9 1.0
```

On peut également sélectionner une partie d'un vecteur à l'aide d'un vecteur de valeurs logiques.

```
v3[v3>0.5] # ne renvoie que les éléments de v3 strictement supérieurs à 0.5
## [1] 0.6 0.7 0.8 0.9 1.0
v3[(v3>0.5)&(v3<1)]
## [1] 0.6 0.7 0.8 0.9
# a renvoyé les éléments de v3 strictement supérieur à 0.5 ET strictement inférieur à 1
v3[(v3<0.5)|(v3==0.9)]
## [1] 0.1 0.2 0.3 0.4 0.9
# a renvoyé les éléments de v3 strictement inférieurs à 0.5 OU égaux à 0.9
```

AUTRES MANIPULATIONS

Recherche d'indice d'un élément :

```
v4 <- c(5, 1, 9, 7, 3, 4, 2.5, 6.32, 8)
which(v4 == 9)
## [1] 3
```

```
which(v4>6)
## [1] 3 4 8 9
which(is.na(v1))
## [1] 3 5
```

Substitution :

```
v0[1:2] <- -3
v0
## [1] -3 -3 3 4 5
v3[v3<0.5] <- 0
v3
## [1] 0.0 0.0 0.0 0.0 0.5 0.6 0.7 0.8 0.9 1.0
v1[is.na(v1)] <- 1000
v1
## [1] -3.0 1.2 1000.0 5.0 1000.0
v2[v2>=10] <- v2[v2>=10]/10
v2
## [1] 1.0 8.0 1.1 7.0 1.2
```

MATRICES

Comme les vecteurs, une matrice est un objet composé d'un ensemble de valeurs toutes du même mode (numérique, logique, etc.). Mais les éléments sont organisés en lignes et en colonnes. Elles possèdent donc l'attribut de dimension (`dim`).

La création peut se faire à l'aide de la fonction `matrix`.

Exemples de création de matrices :

```
matrix(0, 2, 4)
##      [,1] [,2] [,3] [,4]
## [1,]    0    0    0    0
## [2,]    0    0    0    0
matrix(1:6, nrow = 2)
##      [,1] [,2] [,3]
## [1,]    1    3    5
## [2,]    2    4    6
```

```
matrix(1:6, ncol = 2)
##      [,1] [,2]
## [1,]    1    4
## [2,]    2    5
## [3,]    3    6
matrix(1:6, ncol = 2, byrow = FALSE)
##      [,1] [,2]
## [1,]    1    4
## [2,]    2    5
## [3,]    3    6
```

```
cbind(1:3, 2:4)
##      [,1] [,2]
## [1,]    1    2
## [2,]    2    3
## [3,]    3    4
```

```
rbind(1:3, 2:4)
##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    2    3    4
```

FACTEURS

Les facteurs sont des vecteurs particuliers permettant le traitement des données qualitatives. Les facteurs sont de 2 types (comme les variables qualitatives) :

- non ordonnés, par exemple mâle et femelle, appelés **factor** ;
- ordonnés, par exemple grand, moyen, petit, appelés **ordered**.

Création :

- directement par la fonction **factor** :

```
factor(c(1, 22, 1, 1, 2, 2))
## [1] 1 22 1 1 2 2
## Levels: 1 2 22
s <- factor(c("m", "f", "f", "m", "m", NA, "f", "f"))
```

```
s
## [1] m    f    f    m    m    <NA> f    f
## Levels: f m
```

- par conversion d'un vecteur :

```
as.factor(c(1, 22, 1, 1, 2, 2))
## [1] 1 22 1 1 2 2
## Levels: 1 2 22
```

Un attribut des facteurs est `level`.

Exemples de commandes :

```
levels(s)
## [1] "f" "m"
table(s)
## s
## f m
## 4 3
prop.table(table(s))
## s
##      f      m
## 0.5714286 0.4285714
```

```
s2 <- factor(c(1, 0, 0, 2, 1, 0, NA, 1, 1, 2))
levels(s2)
## [1] "0" "1" "2"
nlevels(s2)
## [1] 3
table(s2)
## s2
## 0 1 2
## 3 4 2
```

TABLEAU DE DONNÉES

Un *data frame* est un tableau de données dont toutes les colonnes doivent avoir la même longueur, mais peuvent être de modes différents. C'est la structure de base de R pour le traitement des données statistiques.

La création d'un *data frame* peut s'effectuer en combinant des vecteurs.

Création d'un *data frame* concernant 5 étudiants de L2 Économie contenant leurs choix de module 4 et leur moyenne en L1 :

```
mod <- c("Eco", "Eco", "Gestion", "Eco", "Gestion")
moy <- c(14.2, 11.8, 12.5, 10.4, 13)
df <- data.frame(module = mod, moyL1 = moy)
```

Observons le tableau de données construit avec les commandes suivantes :

```
df
##      module moyL1
## 1      Eco  14.2
## 2      Eco  11.8
## 3 Gestion  12.5
## 4      Eco  10.4
## 5 Gestion  13.0
```

```
str(df)
## 'data.frame':    5 obs. of  2 variables:
## $ module: chr  "Eco" "Eco" "Gestion" "Eco" ...
## $ moyL1 : num  14.2 11.8 12.5 10.4 13
```

```
dim(df)
## [1] 5 2
nrow(df)
## [1] 5
```

```
ncol(df)
## [1] 2
colnames(df)
## [1] "module" "moyL1"
```

THÈME 3. IMPORTATION ET EXPORTATION DE DONNÉES

RÉPERTOIRE DE TRAVAIL

Le répertoire de travail est le répertoire où R lit et écrit ses fichiers.

La commande pour connaître le répertoire de travail actuel est `getwd()`. Pour le changer, on utilise la commande

```
setwd("chemin-vers-le-nouveau-répertoire")
```

On peut également le faire visuellement de la façon suivante : aller dans l'onglet **File** de la partie en bas à droite de **RStudio**, choisir le nouveau répertoire de travail en naviguant dans les dossiers, une fois dans le dossier, cliquer sur **More** et sélectionner **Setting As Working Directory**.

En pratique les données d'une expérience statistique sont enregistrées dans un fichier texte ou csv sous la forme d'un tableau.

IMPORTATION DE DONNÉES

On peut créer un tableau de données en utilisant la fonction `data.frame`, mais le plus souvent on obtient un tableau en important des données à partir d'un fichier existant car en pratique les données d'une expérience statistique sont enregistrées dans un fichier texte ou csv sous la forme d'un tableau.

DONNÉES SIMPLES : `read.table()`

La fonction `read.table` permet de lire et d'importer sous R des données stockées dans un fichier dans un format proche de celui des *data frames*, c'est-à-dire les variables sont arrangées en colonnes et les observations en ligne.

```
Dataframeobtenue <- read.table(file = "fichierdesdonnees.txt")
```

Plusieurs options permettent de paramétrer précisément l'importation de données afin de pouvoir gérer différents formats de tableau.

NOMS DE COLONNES ET DES INDIVIDUS

Souvent la première ligne indique les noms des colonnes. Pour le prendre en compte lors de l'importation de données, on rajoute l'option `header = TRUE` (cette option est par défaut codée `FALSE`). La première ligne n'est alors pas considérée comme des valeurs de mesure, mais comme des noms de variables.

La première colonne peut parfois contenir les noms ou numéros des individus. Pour prendre en compte cela, on utilise l'option `row.names = 1`.

SÉPARATEUR DE CHAMPS

Plusieurs caractères peuvent être utilisés pour séparer les données. Par défaut, la fonction `read.table()` suppose que le séparateur est un espace (incluant plusieurs espaces ou des tabulations). Mais d'autres caractères peuvent être utilisés comme séparateur afin de pouvoir distinguer les valeurs entre elles.

Exemple :

```
Dataframeobtenue <- read.table(file = "fichierdesdonnees.txt", sep = ";")
Dataframeobtenue <- read.table(file = "fichierdesdonnees.txt", sep = ",")
Dataframeobtenue <- read.table(file = "fichierdesdonnees.txt", sep = "\t") #pour la tabulation
```

SÉPARATEUR DÉCIMAL

On peut spécifier le caractère utilisé dans le document comme séparateur décimal classiquement une virgule ou un point à l'aide de l'option `dec`.

DONNÉES ISSUES D'UN TABLEUR : FONCTIONS `read.csv()` OU `read.csv2()`

Quand on souhaite utiliser des données provenant d'un tableur, on commence par les enregistrer au format csv, puis on les importe à l'aide de la fonction `read.csv` ou de la fonctions `read.csv2`. Ces deux fonctions sont identiques à `read.table` sauf pour les options par défaut.

- Pour `read.csv()`, le séparateur de champs est une virgule, le séparateur décimal est un point et par défaut, `header = TRUE`.
- Pour `read.csv2()`, le séparateur de champ est un point-virgule, le séparateur décimal est une virgule et par défaut, `header = TRUE`.

Personnellement je préfère utiliser systématiquement `read.table()` et spécifier les détails après avoir regardé le fichier dans un éditeur de texte.

IMPORTATION ASSISTÉE

Au lieu de taper à la main la ligne de commande permettant l'importation, on peut se faire assister par R. Pour cela, on clique sur l'onglet **Tools**, puis **Import Dataset**, on choisit **From Text File\...** Là une fenêtre s'ouvre pour choisir le fichier. Une fois le fichier choisi, une fenêtre permet de choisir visuellement les différents critères d'importation. Cependant il peut parfois y avoir des problèmes, donc il faut aussi savoir importer à la main.

SPÉCIFICATION DU TYPE DE VARIABLES LORS DE L'IMPORTATION

Notamment pour les chaînes de caractères, il peut être intéressant de spécifier dès l'importation le type de la variable. Par défaut depuis la version 4.0 de R, la commande `read.table()` ne convertit plus automatiquement les chaînes de caractères en facteurs. Cela peut être modifié en jouant avec les arguments `as.is` et `stringsAsFactors`. Pour l'argument `as.is`, il faut spécifier les numéros des variables (chaînes de caractères) que l'on ne souhaite pas voir converties en facteurs. Pour l'argument `stringsAsFactors`, on peut le mettre égal à `TRUE` pour que toutes les variables codées en chaînes de caractères soient considérées comme des facteurs.

CAS DE DONNÉES SOUS FORME D'UN FICHIER `xls` OU `xlsx`

Une première solution est d'ouvrir le fichier avec un tableur puis d'enregistrer au format `csv` les feuilles du tableur qui nous intéressent.

Une autre solution est de lire le fichier directement dans R. Pour cela, une possibilité est d'utiliser le package `readxl` et sa commande `read_excel`.

Il faut d'abord installer le package `readxl`. Ensuite le code est le suivant :

```
library(readxl)
read_excel(path, sheet = NULL, col_names = TRUE,
           range = NULL, skip = 0)
```

Les arguments principaux sont :

- `sheet` : numéro ou nom indiquant la feuille du document à importer ;

- `col_names` : si la première ligne contient les noms des variables, si non on peut aussi les spécifier ;
- `skip` : nombre de lignes à ignorer avant de lire les données, les lignes vides sont automatiquement ignorées ;
- `range` : cellules à lire, par exemple `:B3:D87`.

Attention, l'objet retourné n'est pas un *data frame* classique mais un objet de type `tibble`. Cependant il se manipule de la même façon et peut être transformé en *data frame* classique avec la commande `as.data.frame()`.

AUTRES FORMATS

Il existe beaucoup de formats de données différents mais il existe presque toujours une solution pour les importer dans R :

- le package `haven` pour importer des données provenant de SAS, SPSS ou Stata
- le package `googlesheet4` pour travailler avec des Google Sheets, etc.

EXPORTATION DE DONNÉES

Après avoir travailler sur des données dans R, on peut souhaiter les exporter/enregistrer dans un fichier. Pour cela, on utilise la fonction `write.table`.

```
write.table(objet, file = "nomfichierdesortie", options)
```

Par défaut, `write.table` crée un document avec comme séparateur de colonnes des espaces, comme séparateur décimal le point et il conservent les noms des lignes et des colonnes. Généralement, on utilise

```
write.table(objet, file = "nomfichierdesortie.csv", sep = ";", dec = ".", row.names = FALSE, col.names = TRUE)
```

On ne conserve pas les nom de lignes car très souvent dans les *data frames*, il s'agit simplement de numéro de ligne.

MISE EN PRATIQUE

1. Télécharger les trois fichiers de données présents sur Celene et les mettre dans un répertoire appelé TP4 et spécifier ce répertoire comme répertoire de travail.

```
setwd("TP4")
```

Le chemin dépend de l'emplacement du dossier TP4.

2. Créer un *data frame* appelé `euc0` à partir du fichier `euc0` à l'aide de la commande suivante.

```
euc0 <- read.table("euc0.txt")
```

Vérifier le *data frame* obtenu à l'aide des commandes `head` et `str`.

```
head(euc0, 3)
##      V1 V2 V3 V4
## 1 18.25 36  1  L
## 2 19.75 42  1  L
## 3 16.50 33  1  L
str(euc0)
## 'data.frame':   1429 obs. of  4 variables:
## $ V1: num  18.2 19.8 16.5 18.2 19.5 ...
## $ V2: int   36 42 33 39 43 34 37 41 27 30 ...
```

```
## $ V3: int 1 1 1 1 1 1 1 1 1 1 ...
## $ V4: chr "L" "L" "L" "L" ...
```

3. Même chose avec le fichier `euc1` qui contient en plus les noms des colonnes. Trouver le bon argument pour importer correctement les données.

```
euc1 <- read.table("euc1.txt")
head(euc1, 3)
##      V1    V2    V3    V4
## 1 hauteur circ bloc clone
## 2  18.25   36     1     L
## 3  19.75   42     1     L
```

Les titres de colonnes ne sont pas pris en compte. Pour que ce soit le cas, il faut utiliser l'argument `header`.

```
euc1 <- read.table("euc1.txt", header = TRUE)
head(euc1, 3)
##  hauteur circ bloc clone
## 1  18.25   36     1     L
## 2  19.75   42     1     L
## 3  16.50   33     1     L
str(euc1)
## 'data.frame':    1429 obs. of  4 variables:
## $ hauteur: num  18.2 19.8 16.5 18.2 19.5 ...
## $ circ   : int  36 42 33 39 43 34 37 41 27 30 ...
## $ bloc   : int  1 1 1 1 1 1 1 1 1 1 ...
## $ clone  : chr  "L" "L" "L" "L" ...
```

4. Même chose avec le fichier `euc2`.

```
euc2 <- read.table("euc2.txt", header = TRUE)
head(euc2, 3)
##  hauteur.circ.bloc.clone
## 1      18.25;36;1;L
## 2      19.75;42;1;L
## 3      16.5;33;1;L
str(euc2)
## 'data.frame':    1429 obs. of  1 variable:
## $ hauteur.circ.bloc.clone: chr  "18.25;36;1;L" "19.75;42;1;L" "16.5;33;1;L" "18.25;39;1;L" ...
```

On ne constate qu'une seule variable. Cela est dû au séparateur de champs qui est ici un point-virgule. Il faut donc le spécifier avec l'argument `sep`.

```
euc2 <- read.table("euc2.txt", header = TRUE, sep = ";")
head(euc2, 3)
##  hauteur circ bloc clone
## 1  18.25   36     1     L
## 2  19.75   42     1     L
## 3  16.50   33     1     L
str(euc2)
## 'data.frame':    1429 obs. of  4 variables:
## $ hauteur: num  18.2 19.8 16.5 18.2 19.5 ...
## $ circ   : int  36 42 33 39 43 34 37 41 27 30 ...
## $ bloc   : int  1 1 1 1 1 1 1 1 1 1 ...
## $ clone  : chr  "L" "L" "L" "L" ...
```

5. Modifier l'importation pour que la variable 'clone' soit considérée comme un facteur.

```
euc3 <- read.table("euc2.txt", header = TRUE, sep = ";", stringsAsFactors = TRUE)
str(euc3)
## 'data.frame':    1429 obs. of  4 variables:
## $ hauteur: num  18.2 19.8 16.5 18.2 19.5 ...
## $ circ   : int  36 42 33 39 43 34 37 41 27 30 ...
## $ bloc   : int   1 1 1 1 1 1 1 1 1 1 ...
## $ clone  : Factor w/ 5 levels "D","DH","L","LD",...: 3 3 3 3 3 3 3 3 5 5 ...

euc4 <- read.table("euc2.txt", header = TRUE, sep = ";", as.is = NA)
str(euc4)
## 'data.frame':    1429 obs. of  4 variables:
## $ hauteur: num  18.2 19.8 16.5 18.2 19.5 ...
## $ circ   : int  36 42 33 39 43 34 37 41 27 30 ...
## $ bloc   : int   1 1 1 1 1 1 1 1 1 1 ...
## $ clone  : Factor w/ 5 levels "D","DH","L","LD",...: 3 3 3 3 3 3 3 3 5 5 ...
```

THÈME 4. EXPORTATION DE GRAPHIQUES

CRÉATION D'UN FICHIER CONTENANT UN GRAPHIQUE

On peut créer différents types de document contenant une image créée avec R. Les formats les plus courants sont : pdf, jpeg et png.

La syntaxe de base pour la création d'un pdf est la suivante.

```
pdf(file = "nom_souhaité_du_fichier_du_graphique.pdf")
#code de construction du graphique
dev.off()
```

Pour faire une image au format jpeg, respectivement png, il suffit de remplacer la commande `pdf` par la commande `jpeg`, respectivement `png`, et adapter l'extension du nom de fichier.

```
jpeg(file = "nom_souhaité_du_fichier_du_graphique.jpg")
#code de construction du graphique
dev.off()
```

```
png(file = "nom_souhaité_du_fichier_du_graphique.png")
#code de construction du graphique
dev.off()
```

PERSONNALISATION DU GRAPHIQUE CRÉÉ

Ces commandes possèdent différentes options permettant de personnaliser le document créé.

OPTIONS GÉNÉRALES

Les plus importantes sont `height` et `width` qui permettent de spécifier la hauteur et la largeur de l'image créée. Pour la commande `pdf`, l'unité par défaut est le pouce alors que pour les autres commandes, il s'agit du pixel.

La taille de l'écriture est contrôlée avec l'option `pointsize` qui par défaut vaut 12.

SPÉCIFICITÉS POUR LA COMMANDE `pdf`

Lors de la création d'une image en pdf, on peut également spécifier la taille du document avec l'option `paper` qui avec les arguments `a4` et `a4r`, permet de créer un document pdf au format A4 en portrait ou en paysage.

La commande `pdf` permet également de spécifier la famille d'écriture avec l'option `family` par défaut l'écriture est en Helvetica, d'autres choix possibles sont : Bookman et Palatino.

SPÉCIFICITÉS POUR LA COMMANDE `jpeg`

On contrôle la qualité de l'image créée avec l'option `quality` qui par défaut vaut 75.

```
jpeg(file = "nom_souhaité_du_fichier_du_graphique.jpg",
     width = 1920, height = 1080, quality = 90)
#code de construction du graphique
dev.off()
```

DEUXIÈME PARTIE

TRAVAUX PRATIQUES

PRÉSENTATION DE L'ENSEIGNEMENT

INTERVENANTS

Franck Pillier : franck.pillier@univ-tours.fr, bureau B246 (bâtiment B)

Julie Scholler : julie.scholler@univ-tours.fr, bureau B246 (bâtiment B)

OBJECTIFS

Le but de cet enseignement est de se familiariser avec le logiciel R et d'apprendre à utiliser les outils de gestion de données ainsi que les outils statistiques de base du logiciel R .

CONTENU DES TRAVAUX PRATIQUES

- Découverte du logiciel R : interface RStudio , manipulations et objets de base
- Études descriptives de données enregistrées dans un *data frame*
- Représentations graphiques
- Importation, exportation et manipulation de données

APPROCHE ET SUPPORTS PÉDAGOGIQUES

L'enseignement est composé de 2 séances d'une heure de cours magistral et de 5 séances de travaux pratiques de 2h. Il n'y a pas de séances toutes les semaines, surveillez votre emploi du temps.

Les sujets de TP, ainsi que des corrections partielles, sont disponibles sur mon site perso (<https://juliescholler.gitlab.io>) et via l'ENT sur la page Céline du cours.

MODALITÉS D'ÉVALUATION

- **Session 1** : contrôle continu
- **Session 2** : examen sous forme d'un exercice sur ordinateur

La note de contrôle continu prendra en compte l'évaluation d'exercices à effectuer en dehors des heures de TP et d'une épreuve sur poste informatique en fin de semestre.

PRÉSENCE

La présence en TP est obligatoire. En cas d'absence, vous devez présenter un justificatif ou une justification au chargé de TP dans les 8 jours.

BIBLIOGRAPHIE

Il s'agit de lectures complémentaires aux travaux pratiques. On trouve également de nombreuses ressources sur internet.

- *Statistiques avec R*, Cornillon Pierre-André et Autres (519.5 STA)
- *Le logiciel R* , Lafaye de Micheaux Pierre et Autres (519.5 LAF)
- *Comprendre et réaliser les tests statistiques à l'aide de R* , Millot (519.5 MIL)
- *Initiation à la statistique avec R*, Frédéric Bertrand et Myriam Maumy-Bertrand (519.5 BER)

TP 1. PRÉSENTATION GÉNÉRALE DU LOGICIEL R

Démarrage :

- lancer R avec RStudio ;
- créer un script qui contiendra vos commandes ;
- penser à structurer votre script en mettant les numéros des parties et des exercices en commentaire.

PRINCIPES GÉNÉRAUX DU LOGICIEL R

Commençons par reprendre les commandes vues en cours magistral et dans le polycopié de cours.

R EST UNE CALCULATRICE

Effectuer les commandes suivantes :

```
2*4
3, 5 - 8
3.5 - 8
2.3 + 9 - 5.1
3*2 - 5*(2-4)/6.03
3^2
sqrt(2)
log(2)
round(pi, 2)
```

Si vous ne comprenez pas bien les commandes, utilisez l'aide en tapant `help(sqrt)`, `help(log)` ou `help(round)`. Vous pouvez également accéder à l'aide par l'onglet **help** de la partie en bas à droite de RStudio.

Mise en pratique 1.

Calculer avec R l'expression $e^0 + \frac{\sin(\pi/2)}{\sqrt{4}}$.

CRÉATION D'OBJETS

On peut stocker en mémoire des données, des résultats, etc. Pour cela, on définit des objets R (on reviendra sur les différents types d'objets), à l'aide du symbole `<-` qui permet d'assigner une valeur à un objet.

Par exemple, taper les commandes suivantes.

```
a <- 5
b <- a+1
a
b
```

Mise en pratique 2.

Affecter la valeur 27 à l'objet nommé `x`.

Affecter la valeur 9 à l'objet nommé `X`.

Visualiser les valeurs de `x` et de `X`.

Que constatez-vous ?

Affecter la valeur 5 à l'objet `x`. Que constatez-vous ?

Certains objets existent déjà dans R. Taper `state.area`.
 Pour comprendre ces données, utiliser l'aide `help(state.area)`.

LES VECTEURS

C'est un objet composé d'un ensemble de valeurs toutes du même mode (numérique, logique, etc.). Le nombre d'éléments constitue l'attribut longueur.

CONSTRUCTION

Différentes méthodes sont possibles.

- Construction par la fonction collecteur `c()` :

```
v <- c(10, 4, 5, 8, 3, 2.1, 15, 789, 63, -2)
c(v, 12)
c(TRUE, TRUE, FALSE)
c(1>0, 1 == 1, T, 1<0.5)
```

- Création par l'opérateur séquence `seq()` :

```
seq(1, 8, by = 0.5)
seq(1, 2, length = 5)
```

- Création par la fonction répétition `rep()` :

```
rep(1, 4)
rep("A", 10)
```

- encore une :
`1:5`

CALCULS

Voici quelques exemples de manipulation de vecteurs. Les effectuer.

```
v0 <- 1:5
v0 > 4
sqrt(v0)
sum(v0)
sum(v0 >= 4)
cumsum(v0)
v1 <- c(-3, 1.2, NA, 5, NA)
mode(v1)
is.na(v1)
sum(is.na(v1))
sum(v1)
v1^2
v2 <- c(10, 8, 11, 7, 12)
sort(v2)
v0 + v2
v1 + v2
v0*v2
```

SÉLECTION D'UNE PARTIE D'UN VECTEUR

On peut sélectionner une partie des éléments d'un vecteur en spécifiant les indices des termes nous intéressant. Voici quelques exemples de sélection d'une partie d'un vecteur. Les exécuter et les commenter dans le script.

```
v3 <- seq(0.1, 1, 0.1)
v3[6]
v3[6:8]
v3[c(1, 8, 3, 1)]
v3[-2]
v3[-c(2, 3, 5:8)]
```

On peut également sélectionner une partie d'un vecteur à l'aide d'un vecteur de valeurs logiques.

```
v3[v3>0.5]
v3[(v3>0.5)&(v3<1)]
v3[(v3<0.5)|(v3 == 0.9)]
```

AUTRES MANIPULATIONS

Recherche d'indice d'un élément :

```
v4 <- c(5, 1, 9, 7, 3, 4, 2.5, 6.32, 8)
which(v4 == 9)
which(v4 > 6)
which(is.na(v1))
```

Substitution :

```
v0[1:2] <- -3
v0
v3[v3<0.5] <- 0
v3
v1[is.na(v1)] <- 1000
v1
v2[v2> = 10] <- v2[v2> = 10]/10
v2
```

Mise en pratique 3.

Dans le script, noter les commandes répondant aux questions.

1. Créer le vecteur u composé de 5000 uns.
2. Créer le vecteur v suivant : $(1.3, 2, 5.2, 4.3, 2)$.
3. Créer le vecteur $A = (-10, -9, \dots, 9, 10)$ que vous nommerez **vecA**, à l'aide de la commande `:`. Donner sa longueur directement à partir d'une commande sans calcul.
4. Créer le vecteur $B = (-1.5, -1.4, \dots, 0.5)$ que vous nommerez **vecB**, à l'aide de la commande `seq()`.
5. Créer le vecteur $C = (c_1, \dots, c_n)$ à partir de **vecA** tel que $c_i = 1$ si $a_i < 0$ et $c_i = a_i$ si $a_i \geq 0$,

Mise en pratique 4.

Dans le script, noter les commandes répondant aux questions. Pour la dernière question, répondre sous forme de commentaires.

1. Créer un vecteur à 20 éléments, nommé `alea`, composé de nombres aléatoires extraits d'une loi normale d'espérance 3 et d'écart type 1, en tapant la commande `rnorm(20, mean = 3, sd = 1)` (consulter l'aide `help(rnorm)`).
2. Afficher la valeur du cinquième élément.
3. Afficher les valeurs des 5 derniers éléments.
4. Afficher les valeurs des éléments 1, 4, 8, 12 et 18.
5. Afficher les valeurs inférieures à 2.
6. Créer un nouveau vecteur `aleabis` qui est une copie du vecteur `alea`, puis affecter la valeur 0 aux éléments inférieurs à 3 et la valeur 1 aux éléments supérieurs à 3.
7. Proposer une autre façon de répondre à la question précédente à l'aide de la fonction `ifelse()` (utiliser l'aide `help(ifelse)`).
8. Combien d'éléments de `alea` sont supérieurs à 3? Répéter la commande création du vecteur `alea`. Avez-vous obtenu le même nombre d'éléments supérieurs à 3? Pourquoi?

LES FACTEURS

Création :

- directement par la fonction `factor` :

```
factor(c(1, 22, 1, 1, 2, 2))
s <- factor(c("m", "f", "f", "m", "m", NA, "f", "f"))
s
```

- par conversion d'un vecteur :

```
as.factor(c(1, 22, 1, 1, 2, 2))
```

Un attribut des facteurs est `level`.

Exemples de commandes :

```
levels(s)
table(s)
prop.table(table(s))

s2 <- factor(c(1, 0, 0, 2, 1, 0, NA, 1, 1, 2))
levels(s2)
nlevels(s2)
table(s2)
```

Mise en pratique 5.

On collecte la couleur des yeux de 12 personnes.

1. Créer un facteur `couleurs` regroupant les 12 valeurs obtenues qui sont les suivantes : bleu, marron, vert, marron, marron, bleu, marron, marron, vert, vert, marron, vert.
2. Donner les commandes renvoyant le nombre de modalités de la variable étudiée et l'effectif total.
3. Donner le tableau des effectifs correspondant aux données.
4. Donner une commande renvoyant le tableau des fréquences, à l'aide des fonctions précédentes et d'un calcul.
5. Donner la commande arrondissant le tableau des fréquences au centième.

EXERCICE À RENDRE

(inspiré de Kelly Bodwin)

Enregistrer chaque commande utilisée dans un script nommé `VotreNom-TP1-CR.R` qu'il faudra déposer sur Célène.

Très vite vous allez rencontrer des erreurs dans R. Bien que ce soit frustrant, il ne faut pas en avoir peur. Vous devez juste apprendre dès le début à lire les messages d'erreurs renvoyés par R et repérer les erreurs classiques (fautes de frappe, oubli de virgule, de parenthèse, de guillemets, etc.).

Attention ce n'est pas parce qu'une commande se déroule sans erreur qu'elle a effectué ce que vous souhaitez. À l'aide de la sortie dans la console et de l'environnement, vérifiez régulièrement que vous avez accompli ce que vous souhaitez (ce qui vous est demandé).

Chacun des groupes de commandes suivantes cause des erreurs ou ne fait pas ce qui est demandé. Corrigez-les afin qu'elles réalisent les actions demandées.

1. Créer des vecteurs contenant les lettres capitales, minuscules, les lettres accentuées ou particulières et les signes de ponctuation.

```
minuscules <- c("a", "b", "c", "d", "e", "f", "g", "h", "i", "j", "k",  
               "l", "m", "n", "o", "p", "q", "r", "s", "t", "u", "v",  
               "w", "x", "y", "z")  
  
capitales <- c("A", "B", "C", "D", "E", "F", "G", "H", "I", "J", "K",  
              "L", "M", "N", "O", "P", "Q", "R", "S", "T", "U", "V",  
              "W", "X", "Y", "Z")  
  
speciales <- c(é, è, ê, à, ù, â, î, ô, ç)  
  
ponctuation <- c(".", ",", "!", "?", "'", '"', "(", ")", " ", "-", ";", ":")
```

2. Créer un vecteur contenant les entiers consécutifs de 1 à 200.

```
nombres <- 1;200
```

3. Créer un long vecteur contenant tous les symboles.

```
symboles <- c(minuscules, Capitales, spéciales, PONCTUATION)
```

4. Vérifier qu'il n'y a pas de valeurs manquantes dans le vecteur `symboles` créé.

```
sum(isna(symboles))
```

5. J'ai codé un message dans le vecteur `mot_code`. Le décoder en corrigeant les commandes suivantes.

```
mot_code <- c(28, 9, 5, 14, 70, 10, 15, 21, 5, 70, 64)  
paste(symboles(mot_code), collapse = "")
```

TP 2. DATA FRAMES ET STATISTIQUES UNIVARIÉES QUANTITATIVES

Objectifs du TP :

- découvrir la classe `data.frame` ;
- réaliser des graphiques simples ;
- soigner la présentation de graphiques ;
- déterminer les paramètres d'une série statistique.

EXERCICE 1.

La création d'un *data frame* peut s'effectuer en combinant des vecteurs.

1. Créer le *data frame* concernant 4 individus contenant leur sexe, leur taille en *cm* et leur poids en *pounds* avec les commandes suivantes :

```
s <- c("M", "F", "F", "F")
t <- c(182, 165, 159, 171)
p <- c(164, 115, 140, 147)
df <- data.frame(sexe = s, taille = t, poids = p)
```

2. Observer le tableau de données construit avec les commandes suivantes :

```
df
str(df)
dim(df)
nrow(df)
ncol(df)
colnames(df)
```

3. Construire un tableau de données nommé `data` concernant les individus précédents contenant leur sexe, leur taille en *m*, leur poids en *kg* et leur IMC (arrondi à un chiffre après la virgule). L'indice corporel s'obtient de la façon suivante :

$$\text{IMC} = \frac{\text{Poids en kg}}{\text{Taille en m}^2}$$

EXERCICE 2.

On va travailler sur l'objet `InsectSprays` déjà existant dans R. Il contient une variable quantitative discrète que nous allons étudier.

1. Avant de manipuler cet objet, consultons sa description dans l'aide fournie par R. Pour cela, vous pouvez aller dans l'onglet *Help* de RStudio ou taper l'une des commandes suivantes (observez leurs différences).

```
??insectspray
help(InsectSprays)
?InsectSprays
```

2. Observer l'objet `InsectSprays` avec les commandes suivantes :

```
df <- InsectSprays
str(df)
head(df)
```

Quelle est l'utilité de la commande `head` ?

3. Effectuer les commandes suivantes.

```
count
df$count
```

4. Tableaux des effectifs, des fréquences et des fréquences cumulées de la variable quantitative discrète `count`.

- Réaliser le tableau des effectifs de la variable `count` avec la commande suivante.

```
table(df$count)
```

- Réaliser le tableau des fréquences en consultant l'aide de la fonction `prop.table`. Pour plus de lisibilité, arrondir à 3 chiffres après la virgule.
- Construire le tableau des fréquences cumulées.

5. Représentations graphiques.

- Réaliser un premier graphique grâce à la commande suivante.

```
plot(df$count)
```

Essayer la commande suivante. Que permet l'argument `pch` ?

```
plot(df$count, pch = 4)
```

- Représenter graphiquement les effectifs par modalité avec la commande suivante.

```
plot(table(df$count), main = "Effectifs", xlab = "Nombre d'insectes sur chaque parcelle",
ylab = "Nombre de parcelles")
```

- Représenter la courbe des fréquences cumulées. Utiliser l'argument `type` afin d'obtenir le graphique approprié. Faire attention aux abscisses.
- On peut ajouter des droites et du texte sur un graphique. Essayer les commandes suivantes.

```
plot(df$count, pch = 4, main = "Nombre d'insectes sur chaque parcelle",
xlab = "Numéro de parcelle", ylab = "Effectif")
abline(h = 10, col = "red")
text(x = 30, y = 10, "seuil")
```

- Consulter l'aide de la commande `text` et utiliser l'argument adéquat pour que le mot `seuil` ne soit plus placé sur la droite sans modifier les coordonnées (30,10).
- (*plus dur*) On peut tracer plusieurs droites d'un coup avec la commande `abline`. Effectuer les commandes suivantes et essayer de les comprendre.

```
plot(df$count, pch = 4, main = "Nombre d'insectes sur chaque parcelle",
     xlab = "Numéro de parcelle", ylab = "Effectif")
abline(h = 10, col = "red")
text(x = 30, y = 10, pos = 3, "seuil", col = "darkred")
abline(v = 12*1:5+0.5, col = "blue")
text(x = 12*0:5+5.25, y = rep(25, 6), c("A", "B", "C", "D", "E", "F"), col = "blue")
```

EXERCICE 3.

Dans cet exercice, nous allons travailler avec le tableau de données **trees** concernant des cerisiers noirs. Il contient les informations suivantes : **Girth** (circonférence), **Height** (hauteur) et **Volume**. Les unités sont anglo-saxonnes, mais cela n'a aucune incidence pour la suite.

1. Commencer par affecter **trees** dans un nouvel objet nommé **data** et observer son contenu à l'aide de commandes déjà rencontrées.
2. Effectuer les commandes ci-dessous et les décrire en commentaire.

```
summary(data)
mean(data)
mean(data$Height)
quantile(data$Height, 0.25)
IQR(data$Girth)
```

3. Trouver une commande permettant d'obtenir d'un coup tous les quartiles, puis une autre permettant d'obtenir tous les déciles.
4. Réaliser un histogramme avec la commande suivante.

```
hist(data$Height)
```

Mettre un titre principal et modifier les titres des axes.

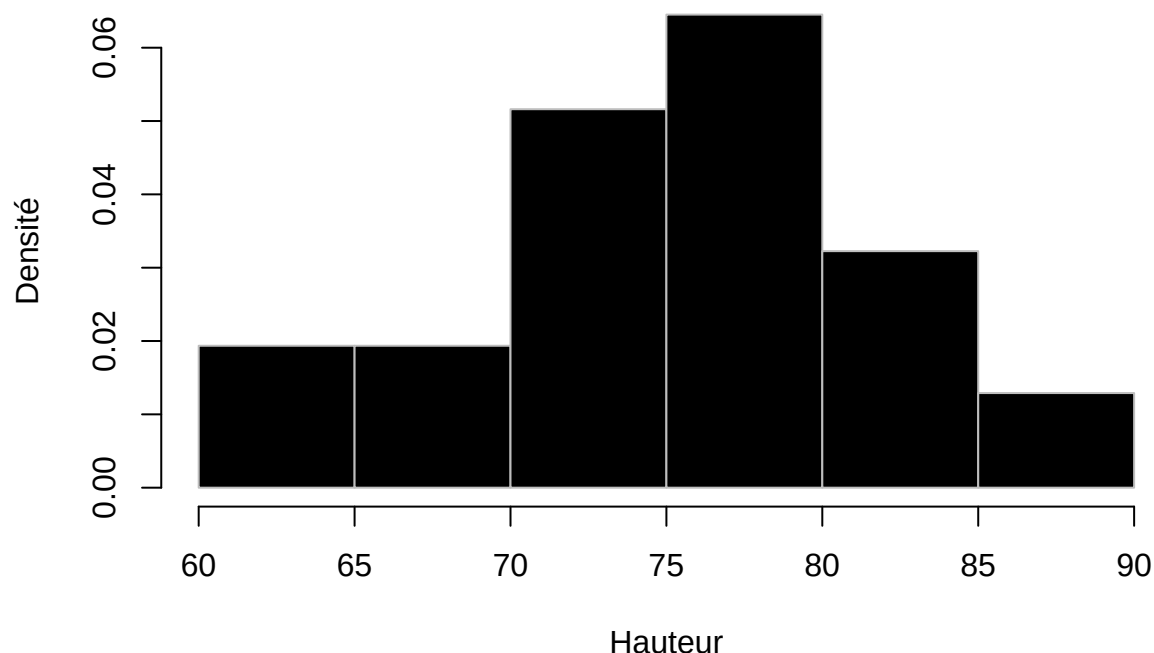
5. Observer les différentes informations contenues dans l'objet **Histo**.

```
Histo <- hist(data$Height, plot = FALSE)
Histo
```

Que contiennent **Histo\$breaks** et **Histo\$counts** ?

6. Déterminer la commande qui permet d'obtenir la représentation graphique ci-contre :

Répartition des arbres en fonction de leur hauteur



7. Effectuer les commandes ci-dessous et ajouter le bon titre pour l'axe des ordonnées.

```
bornes <- c(60, 70, 75, 80, 90)
hist(data$Height, main = "Répartition des arbres \n en fonction de leur hauteur",
      xlab = "Hauteur", ylab = "", breaks = bornes)
```

EXERCICE 4 (POUR ÉVALUATION).

Enregistrer chaque commande utilisée dans un script nommé **VotreNom-TP2-CR.R** qu'il faudra déposer sur Célène. Les questions commençant par (*) sont un peu plus dures. Vous pouvez les laisser de côté lors de la première tentative ou si vous avez des difficultés.

Vous allez travailler sur les données contenues dans la base de données **ToothGrowth**, existante par défaut dans R.

Dans cette base de données sont contenues des informations sur 60 cochons d'Inde ayant reçu de la vitamine C par administration journalière. Les variables présentes sont :

- **len** : la longueur de leurs odontoblastes (cellules responsables de la croissance des dents) ;
- **dose** : la dose journalière reçue en mg/jour ;
- **supp** : la méthode d'administration qui est soit via du jus d'orange codé **OJ**, soit via de l'acide ascorbique, une forme de vitamine C, codée **VC**.

1. Affecter les données dans un objet nommé **data**.
2. Donner l'étendue de la longueur des odontoblastes des cochons d'Inde.
3. Effectuer un histogramme de la longueur des odontoblastes des cochons d'Inde, soigné (titre, couleurs).
4. Réaliser un tableau des effectifs du dosage de vitamine C reçu.
5. Représenter graphiquement et de façon soignée et adaptée la courbe des fréquences cumulées du dosage de vitamine C reçu par jour.
(*) Travailler le graphique pour que l'on voit la courbe commencer à 0 et continuer après 1.
6. Représenter graphiquement la longueur des odontoblastes en fonction du dosage journalier en vitamine C. Pour améliorer la lisibilité du graphique, changer le type de point.

(*) Vous pouvez aussi chercher comment faire pour les points soient colorés en fonction du type de supplémentation.

7. Ajouter sur le graphique précédent deux droites horizontales une au niveau de la longueur moyenne des ondoblastes et l'autre au niveau de la valeur médiane des ondoblastes. Bien distinguer les droites par des couleurs différentes et du texte.
-

TP 3. CALCUL MATRICIEL ET TABLEAUX DE CONTINGENCE

Objectifs du TP :

- découverte de la classe `matrix`;
- calcul matriciel;
- manipulation de tableaux de contingence;
- représentations graphiques de la distribution d'une variable qualitative.

EXERCICE 5.

Comme un vecteur, une matrice est un objet composé d'un ensemble de valeurs toutes du même mode (numérique, logique, etc.). Mais les éléments sont organisés en lignes et en colonnes.

1. La création peut se faire à l'aide de la fonction `matrix`.
Créer et observer les matrices A, B et C avec les commandes suivantes.

```
A <- matrix(1:6, nrow = 2)
A
B <- matrix(1:6, ncol = 2)
B
C <- matrix(1:6, ncol = 2, byrow = T)
C
```

2. Tester les commandes suivantes. Que renvoient-elles ?

```
length(A)
dim(A)
length(B)
dim(B)
```

3. On peut également construire des matrices en combinant des vecteurs.
Tester les commandes suivantes.

```
v1 <- c(25, -12, 8)
v2 <- c(4.5, 87, -3.2)
cbind(v1, v2)
```

4. Trouver la commande permettant de combiner `v1` et `v2` afin d'obtenir la matrice $\begin{pmatrix} 4.5 & 87 & -3.2 \\ 25 & -12 & 8 \end{pmatrix}$.

5. Réaliser les calculs suivants et noter en commentaire leurs actions.

```
A+10
A-20
A*10
```

	Blond	Roux	Châtain	Brun	Noir
Garçon	592	119	849	504	36
Fille	544	97	677	451	14

```

A/10
A+B
B+C
A*B
B*C
A%%B
B%%C
A%%t(A)
A%%A
A^2
sqrt(A)

```

EXERCICE 6.

1. Créer la matrice A à l'aide de la commande suivante.

```
A <- matrix(c(23, 34, 31, 46), ncol = 2)
```

2. Vérifier que la matrice A est inversible en calculant son déterminant. Calculer son inverse à l'aide de la commande `solve()`.
3. Vérifier que le produit $A \times A^{-1}$ est égal à la matrice identité.

4. Créer la matrice B égale à $\begin{pmatrix} 3 & 4 & 2 \\ 1 & 3 & 2 \\ 1 & 2 & 1 \end{pmatrix}$ et calculer le produit de B avec son inverse. Commenter.

5. À l'aide des commandes vues précédemment, résoudre le système suivant $\begin{cases} 23x + 31y = 1 \\ 34x + 46y = 2 \end{cases}$.

6. Résoudre directement l'équation de la question précédente en une seule commande à l'aide de la fonction `solve` (consulter l'aide).

EXERCICE 7.

On souhaite étudier le lien entre la couleur des cheveux et le sexe à partir d'un exemple historique dû à FISHER. Il a répertorié la couleur des cheveux de garçons et de filles d'un district écossais.

1. Construire une matrice, nommée `tab`, contenant les effectifs de la même forme que le tableau précédent. Puis ajouter des noms aux lignes et aux colonnes avec les commandes suivantes.

```
rownames(tab) <- c("Garçon", "Fille")
colnames(tab) <- c("Blond", "Roux", "Châtain", "Brun", "Noir")
```

2. Sélection de parties.

- Tester les deux commandes suivantes.

```
tab[2, 1]
tab[1, 2]
```

- Extraire de `tab` un vecteur contenant les effectifs des garçons roux et bruns et un autre contenant les effectifs des roux et des rousses.
3. Utiliser la commande `margin.table` pour fournir les distributions marginales en effectifs.

```
margin.table(tab, 1)
margin.table(tab, 2)
```

4. Construire le tableau des fréquences arrondies à 3 chiffres après la virgule. Ajouter les marges et changer les titres des marges de `Sum` en `Total`.
 5. Donner les fréquences conditionnelles des couleurs de cheveux pour chacun des sexes.
 6. Représentations graphiques.
- Représenter la répartition des couleurs de cheveux des garçons avec la fonction `plot` en utilisant l'argument `lwd` pour élargir les bâtons. Utiliser également le vecteur défini ci-dessous pour colorer les différents bâtons.

```
couleur <- c("Gold", "OrangeRed", "Goldenrod", "Brown", "Black")
```

- On peut également utiliser la commande `barplot` pour ce type de graphique. L'utiliser pour la répartition des couleurs de cheveux des filles, faire intervenir l'argument `space` pour espacer un peu plus les rectangles produits.
- Quel est le principal avantage de la commande `barplot` ?
- Une autre représentation graphique est le diagramme circulaire (il y a quasiment toujours mieux à faire qu'un diagramme circulaire mais il faut quand même savoir en faire un si on ne nous laisse pas le choix). Essayer la commande suivante.

```
pie(tab[1, ])
```

Travailler ce graphique en modifiant les couleurs. Essayer de trouver une façon d'ajouter les pourcentage sur le graphique.

EXERCICE 8 (À RENDRE).

(inspiré de Kelly Bodwin)

L'objectif de cet exercice est de décoder un message codé que je vous adresse.

Commencer par charger le message codé avec cette ligne de code. *Attention vous devez être connecté à internet pour que cette commande fonctionne.*

```
top_secret <- read.csv("https://juliescholler.gitlab.io/publication/l2r1-2122/message_code.csv",
```

Pour décoder le message, vous devez effectuer correctement les instructions ci-dessous.

1. Ajouter 14 à chacun des nombres
2. Multiplier tous les nombres par 18, puis soustraire 257 à tous les nombres.
3. Appliquer la fonction exponentielle à tous les nombres.
4. Mettre tous les nombres au carré.

Vérification : à ce niveau vous devriez avoir ... nombres strictement inférieurs à 20.

5. Transformer votre vecteur en une matrice à 5 colonnes remplies colonne par colonne.

6. Indépendamment du message codé, créer un vecteur nommé `pairs` contenant les entiers pairs compris entre 1 et $2 * (\text{length}(\text{top_secret}) / 5)$.
7. Soustraire le vecteur `pairs` à la première colonne de la matrice.
8. Soustraire 100 à tous les nombres de la troisième colonne situés entre la 18ème et la 24ème lignes (comprises).
9. Multiplier par 2 tous les nombres des colonnes 2 et 4.
10. Retransformer votre matrice en vecteur en lisant les valeurs colonne par colonne.

Vérification : à ce moment, tous les nombres d'indice supérieur à ... devraient être inférieurs à ...

11. Prendre la racine carrée de tous les nombres d'indice entre 57 et 240 (inclus).
12. Arrondir tous les nombres à l'entier le plus proche.
13. Remplacer toutes les occurrences du nombre 77 par 12.

Vous être presque prêt · e à décoder le message.

14. Avant cela, créer le vecteur suivant contenant les symboles possibles.

```
accents <- c("â", "é", "è", "ê", "î", "û", "À", "É", "È", "Ê", "Î", "Û")
ponctuation <- c(".", ",", "!", "?", "'", '"', "(", ")", " ", "-", ";", ":")
symboles <- c(letters, LETTERS, accents, ponctuation)
```

15. Et maintenant décoder avec la commande suivante.

```
msg <- paste(symboles[top_secret4], sep = "", collapse = "")
msg
```

TP 4. IMPORTATION DE DONNÉES ET VARIABLES QUALITATIVES

Objectifs du TP :

- manipulation des objets de type **factor** ;
- construction et exportation de graphiques adaptés aux données qualitatives.

EXERCICE 9.

1. Créer et manipuler les facteurs suivants (levels, tableaux d'effectifs).

```
f1 <- factor(c(1, 22, 1, 1, 2, 2))
f2 <- factor(c("m", "f", "f", "m", "m", NA, "f", "f"))
```

2. On collecte l'information sur la couleur des yeux de 12 personnes.

- Créer un facteur **couleurs** regroupant les 12 valeurs obtenues : bleu, marron, vert, marron, marron, bleu, marron, marron, vert, vert, marron, vert.
- Donner les commandes renvoyant le nombre de modalités de la variable étudiée et l'effectif total.
- Construire le tableau des fréquences arrondies au centième.

EXERCICE 10.

Dans cet exercice, on va travailler les représentations graphiques adaptées aux situations où des variables qualitatives interviennent.

On va utiliser les données **euc2**. Les données contiennent la hauteur (**hauteur**) et la circonférence (**circ**) de différents types d'eucalyptus (**clone**) plantés dans différents sols (**bloc**).

1. Importer les données dans une *data frame* nommé **df**.
2. Donner la structure et un résumé (moyenne, etc.) des différentes données. Commenter.
3. Modifier la commande d'importation pour que la variable **clone** soit directement considérée comme un facteur.
4. Représenter la hauteur en fonction du sol avec la commande `plot(df$hauteur~df$bloc)`. Que pensez-vous de cette représentation graphique ?
5. Les variables *type de sol* et *clone* sont-elles qualitatives ou quantitatives ? Quel est leur mode dans R ?
6. Quand on souhaite représenter une variable quantitative en fonction d'une variable qualitative, une série de diagrammes en boîte est bien adaptée. Cela se fait automatiquement avec la fonction **plot** quand la variable explicative est considérée comme un facteur.
Effectuer la commande suivante.

```
plot(df$hauteur~df$clone)
```

On peut forcer la représentation en diagramme en boîte avec la fonction **boxplot** ou faire en sorte que R considère la variable en tant que facteur.

- Effectuer des diagrammes en boîte de la hauteur en fonction du type de sol à l'aide de la fonction `boxplot`.
- Rajouter à la commande `boxplot` précédente les options suivantes : `range = 0` et `boxwex = 0.5`. Que font ces deux options ?
- Changer le type de la variable `bloc` de `numeric` à `factor` avec la commande suivante.

```
df$bloc <- as.factor(df$bloc)
```

- Renommer les levels de la nouvelle variable `bloc` avec la commande suivante.

```
levels(df$bloc) <- c("sol_A", "sol_B", "sol_C")
```

- Représenter la circonférence en fonction du type de sol avec la commande `plot`.
7. Afin de visualiser la proportion de chaque clone et de chaque type de sol, réaliser deux diagrammes.
 8. Comme les clones autres que DH ne sont pas beaucoup représentés, nous allons fusionner tous les levels de clones autres que le level DH avec les commandes suivantes.

```
levels(df$clone)
levels(df$clone) <- c("autres", "DH", "autres", "autres", "autres")
str(df$clone)
```

8. Représenter uniquement les hauteurs pour les clones qui ne sont pas du type DH avec les commandes suivantes.

```
filtre_autre <- df$clone == "autres"
plot(df$hauteur[filtre_autre]~df$bloc[filtre_autre], xlab = "Type de sol",
     ylab = "Hauteur des clones de type non DH")
```

9. Créer un nouveau *data frame* intitulé `eucAB` ne contenant que les individus des sols A et B et observer le nombre de levels de la variable `bloc` ainsi que son tableau d'effectifs.
10. Utiliser la commande `droplevels` pour améliorer la variable `bloc`.
11. Utiliser la commande `table` pour construire le tableau de contingence des types de sols en fonction des différents clones.

EXERCICE 11 (POUR ÉVALUATION).

Vous allez travailler sur des données de dépenses de clients dans un supermarché. Elles sont contenues dans le fichier `clients-supermarche.csv` sur Celene. Ce jeu de données contient les informations suivantes :

- **Gender** : le genre de l'individu ;
 - **Age** : son âge ;
 - **AnnualIncome** : son revenu annuel en millier de dollars ;
 - **SpendingScore** : son niveau de dépenses codé entre 0 et 100.
1. Importer les données dans un *data frame* intitulé `dep`, en faisant en sorte que la variable **Gender** soit considérée comme un facteur.
 2. Pour combien d'individus la variable **SpendingScore** est mal codée ? Supprimer ces individus de la base de données.
 3. Ajouter une nouvelle variable **NiveauDep**, au *data fame* `dep` qui vaut :
 - **peu** si les dépenses du client correspondent à un score inférieur strictement à 25 ;
 - **moyen** si les dépenses du client correspondent à un score entre 25 et 70 ;

- beaucoup si les dépenses du client correspondent à un score supérieur ou égal à 70.
4. Donner le tableau de contingence avec marges des niveaux de dépenses et du genre des individus.
 5. Représenter sous forme de diagramme en bâtons la répartition des âges des clients.
 6. Représenter trois graphiques, un en-dessous de l'autre, représentant pour le premier la distribution des revenus des clients de genre féminin, pour le second la distribution de revenus des clients de genre masculins et pour le dernier ces deux distributions mais sur un même graphique.
Enregistrer, via des lignes de commandes, l'ensemble de graphiques produit dans un document pdf au format A4 portrait, nommé **VotreNom-genre.pdf**.
Utiliser un maximum de paramètres pour soigner l'image produite.
 7. Réaliser un nuage de points du niveau de dépenses en fonction du revenu.
Enregistrer, via des lignes de commandes, ce graphique produit dans un document jpeg, nommé **VotreNom-revenu.jpg**.
Utiliser un maximum de paramètres pour soigner l'image produite.
 8. Créer un nouveau *data frame* intitulé **clientes** ne contenant que les données des clients s'étant déclarés de genre féminin et enlever les variables et modalités inutiles.
 9. Exporter le tableau de données ainsi créé dans un fichier nommé **clientes.csv** contenant les noms des variables, ne contenant pas les noms des individus et dont le séparateur de colonnes est la virgule.
 10. Déposer sur Célène –uniquement– le script, intitulé **VotreNom-TP4.R** contenant toutes les commandes, y compris celles de créations des fichiers pdf, jpeg et csv à exporter.
-

TP 5. MANIPULATIONS AVANCÉES DE DATA FRAME

Objectifs du TP :

- extraire des individus et/ou des variables ;
- modifier/créer des variables ;
- déterminer les paramètres de sous-groupes.

EXERCICE 12.

Les données étudiées sont issues d'une enquête réalisée au début des années 80, concernant les étudiants américains. Elles ont été utilisées pour étudier l'effet des universités communautaires sur le niveau scolaire. Les données sont contenues dans le fichier `student.txt` qui contient les informations suivantes :

- `female` : 1 = *female* / 0 = *male* ;
- `black` : 1 = *black* / 0 = *not black* ;
- `hispanic` : 1 = *hispanic* / 0 = *not hispanic* ;
- `dadcoll` : 1 = père diplômé du supérieur / 0 = père non diplômé du supérieur ;
- `momcoll` : 1 = mère diplômée du supérieur / 0 = mère non diplômée du supérieur ;
- `ownhome` : 1 = famille propriétaire de son logement / 0 = famille non propriétaire ;
- `urban` : 1 = école en zone urbaine / 0 = école en zone rurale ;
- `dist` : distance université-domicile en dizaine de miles ;
- `tuition` : frais de scolarité en milliers de \$;
- `ed` : nombre d'années de scolarisation ;
- `incomehi` : 1 = revenu familial annuel > \$25, 000 / 0 = revenu familial annuel ≤ \$25, 000 ;
- `county` : numéro de code du comté ;
- `state` : numéro de code de l'état.

1. Importer les données du fichier `student.txt` dans un *data frame* intitulé `stu` et l'observer.
2. Effectuer les commandes ci-dessous et les décrire.

```
mean(stu$tuition)
mean(stu$tuition[stu$female == 1])
mean(stu$tuition[stu$ed >= 14])
mean(stu$tuition[!(stu$ed < 14)])
quantile(stu$dist[stu$urban == 1], 0.25)
sd(stu$dist[stu$hispanic == 1&stu$tuition > 1])
```

3. Déterminer la médiane du nombre d'années d'étude des femmes dont la mère est diplômée du supérieur ainsi que le troisième quartile du nombre d'années d'étude des hispaniques dont les frais de scolarité sont supérieurs à 1000 \$.
4. Créer une commande qui permet de vérifier qu'aucun étudiant n'ait été codé simultanément *black* et *hispanic*.
5. Effectuer les commandes suivantes et expliquer ce qu'elles produisent.

```

tab1 <- stu[, c(8:10)]
head(tab1)
tab2 <- stu[-c(1, 3), ]
head(tab2)
tab3 <- stu[stu$female == 1, ]
head(tab3)
tab4 <- tab3[, -1]
head(tab4)
tab5 <- stu[stu$female == 1, -1]
head(tab5)
tab6 <- stu[stu$county == 28 & stu$tuition >= 1.2, -(1:3)]
head(tab6)

```

6. Extraire dans un *data frame* intitulé **tab7** les étudiants hispaniques dont le revenu familial est supérieur à 25.000 \$ et supprimer les variables **hispanic**, **black** et **incomehi**. Combien y a-t-il d'individus ?
7. Convertir la distance université-domicile en kilomètres.
8. On souhaite créer une variable simplifiée sous forme d'un facteur codant les frais de scolarité de la façon suivante :
 - **faible** : si les frais de scolarité sont inférieurs ou égaux à 600 \$;
 - **moyen** : s'ils sont compris strictement entre 600 \$ et 1000 \$;
 - **eleve** : s'ils sont supérieurs ou égaux à 1000 \$.

Pour cela, commencer par créer un vecteur **tuition.fees**, en répétant la modalité **moyen**, puis corriger (en deux temps) les modalités correspondant aux autres situations.

Terminer en donnant le tableau des effectifs correspondant.

9. On souhaite ajouter au *data frame* les résultats de ces étudiants au *Base Year Composite Test Score*, un test évaluant le niveau de l'étudiant. Pour cela, importer les données du fichier **bytest.txt** et créer un nouveau *data frame* intitulé **stubis** contenant les données de **stu** et celle de **bytest**.
10. Construire un vecteur intitulé **commu** permettant d'identifier la communauté à laquelle appartient l'étudiant, en codant **Hispanic** pour *hispanic*, **Black** pour *black* et **White** pour ni *black*, ni *hispanic*. L'ajouter au *data frame*.
11. Pour chacune des communautés, déterminer la moyenne du *bytest* avec la commande suivante.

```
by(stubis$score, commu, mean)
```

13. En utilisant la commande **by** et son aide, renvoyer les quartiles des *bytest* par sexe.
14. On souhaite obtenir les écarts type des *bytest* par sexe. Cependant la commande **sd** renvoie l'écart type corrigé. On peut créer la fonction **et** calculant l'écart type avec le commande suivante.

```

et <- function(x) {sqrt( (length(x)-1)/length(x) )*sd(x)}
et(stubis$score)

```

L'utiliser pour renvoyer les écarts type des *bytest* par sexe.

15. Construire une nouvelle variable intitulée **par.sup** qui vaut 0 si aucun parent n'est diplômé du supérieur et 1 sinon. Renvoyer les déciles du *bytest* pour ces deux groupes.

EXERCICES D'ANNALES

Voici quelques extraits de contrôle sur poste ou de session 2. Il n'y a et n'aura pas de correction. Par contre, vous avez bien sûr le droit de nous poser des questions et de nous montrer votre travail sur ces exercices d'annales pour voir si cela correspond bien à ce qui est attendu.

EXERCICE 13 (EXTRAIT DE L'EXAMEN DE SESSION 2 - 2014-2015)

Le fichier de données se nomme `session2-1415.txt` et se trouve sur Celene. Il contient les données brutes d'un sondage réalisé auprès d'individus regardant quotidiennement la télévision. On a répertorié :

- le genre de l'individu `Genre` ;
 - le nombre de personnes dans le foyer `Foyer` ;
 - le diplôme obtenu `Diplome` (SupBac+2 signifie supérieur à Bac +2) ;
 - la durée moyenne quotidienne devant la télévision, `Duree`, en minutes.
1. Importer les données dans un *data frame* intitulé `data`.
 2. Déterminer la taille de l'échantillon et la structure de `data`. Préciser les types des variables.
 3. Donner un résumé des variables. Donner la liste des diplômes par une commande spécifique.
 4. Effectuer l'histogramme représentant la durée quotidienne devant la télévision.
 5. Calculer le nombre de personnes qui regardent 2h ou plus par jour en moyenne la télévision.
 6. Extraire dans un *data frame* intitulé `dataF` les données concernant les femmes.
 7. Supprimer la colonne `Genre` de `dataF`.
 8. Effectuer les diagrammes en boîte de la durée quotidienne devant la TV en fonction du nombre de personnes dans le foyer. Soigner le graphique.
 9. Déterminer la moyenne arrondi au dixième de la durée quotidienne devant la TV pour les individus appartenant à un foyer comportant 4 personnes.
 10. Créer un vecteur intitulé `Vec` qui prend la valeur 1 si l'individu possède le bac ou un diplôme supérieur, et qui prend 0 sinon.
 11. Calculer la proportion, arrondie au millième, d'hommes qui ont le bac ou plus dans l'échantillon .
 12. Déterminer le tableau de contingence du couple de variables `Genre` et `Diplome`.

EXERCICE 14 (EXTRAIT DE L'EXAMEN DE SESSION 2 - 2016-2017)

Vous allez travailler sur des données concernant des maladies cardiaques. Elles sont contenues dans le fichier `maladiecardiaque_homme.csv` présent sur Celene. Ce tableau de données contient les variables suivantes :

- `age` : l'âge du patient ;
- `chest_pain` : le type de douleur à la poitrine ;
- `rest_bpress` : la pression sanguine au repos ;
- `blood_sugar` : la présence d'un taux élevé de sucre dans le sang ;
- `rest_electro` : les anormalités détectées lors de la réalisation d'un électrocardiogramme au repos ;
- `max_heart_rate` : le rythme cardiaque maximal ;
- `exercice_angina` : la présence d'une angine de poitrine suite à un effort ;
- `disease` : présence d'une maladie cardiaque.

1. Importer les données dans un *data frame* intitulé **maladie**.
2. Représenter graphiquement les proportions des différents types de douleurs à la poitrine.
3. Représenter graphiquement le rythme cardiaque maximal en fonction de l'âge.
4. Réaliser un histogramme des rythmes cardiaques maximums pour les patients ayant eu une angine de poitrine et un autre pour ceux n'en ayant pas eu.
Veiller à réaliser ces histogrammes de façon à ce qu'ils soient facilement comparable (échelles).
5. Quel est le rythme cardiaque moyen des personnes malades ?
6. Donner en une unique commande les différents quartiles de la pression sanguine des personnes non malades.
7. Déterminer le nombre de patients ayant un taux de sucre dans le sang élevé.
8. Créer la variable **high_rate** valant **oui** si le rythme cardiaque maximal est supérieur à 150 et **non** sinon.
9. Extraire dans un *data frame* intitulé **malade** les données des individus malades (sans la variable **disease**).

EXERCICE 15 (EXAMEN DE SESSION 1 - 2019-2020)

Vous allez travailler sur des données concernant des logements vendus, au cours de 8 années, dans Saragota County dans l'État de New York.

Elles sont contenues dans le fichier **saragotaA.csv** sur Celene. Dans ce fichier, on a les informations suivantes :

- **prix** : prix de vente en dollars ;
 - **bord_mer** s'il est ou non en bordure de mer ;
 - **age** : l'âge du logement au moment de la vente ;
 - **chauffage** : le type de chauffage ;
 - **superficie** : la superficie habitable en *square foot* ;
 - **chambres** : le nombre de chambres du logement ;
 - **cheminees** : le nombre de cheminées ;
 - **sdb** : le nombre de salle d'eau ou de bain ;
 - **pieces** : le nombre de pièces ;
 - **psce_cheminee** : la présence de cheminées.
1. Importer les données du fichier **saragotaA.csv** dans un *data frame* intitulé **df**.
 2. Quel est le prix moyen des logements vendus ?
 3. Transformer la surface habitable actuellement en *square foot* en mètre carré, arrondi à l'unité : 1 mètre carré = 10.76 sq ft.
 4. Représenter par des diagrammes en boîte la répartition des âges des logements selon qu'ils soient en bordure de mer ou non, et ajouter sur le graphique une droite horizontale colorée au niveau de l'âge moyen de tous les logements ainsi qu'un texte de la même couleur spécifiant de quoi il s'agit.
 5. Ajouter au *data frame* une variable nommée **neuf** qui vaut **Oui** si l'âge du logement est 0 et **Non** sinon.
 6. Donner le tableau de contingence entre la présence de cheminée et le fait que le logement soit neuf.
 7. Donner le premier quartile de la surface habitable des logements en bordure de mer.
 8. Vérifier que pour tous les logements, il y a plus de pièces que de chambres de renseignées, en donnant le nombre de logements qui ne vérifient pas cela.
 9. Créer un document **pdf**, au format A4, intitulé **Nom-graph.pdf**, contenant deux histogrammes un au-dessus de l'autre de la répartition des prix des logements : un pour ceux possédant un chauffage électrique et le second pour ceux ne possédant pas un chauffage électrique. *La comparaison des prix entre les deux graphiques doit être aisée.*
 10. Combien de logements vendus à moins de 150 000 dollars ne possèdent pas de cheminée ?
 11. Vérifier que les variables concernant la présence de cheminée sont cohérentes, en donnant le nombre de logements pour lesquels le codage n'est éventuellement pas correct.
 12. Extraire un *data frame* ne contenant que les logements neufs et supprimer de ce *data frame* toutes les variables et/ou *levels* devenus inutiles.
 13. Sauvegarder ce *data frame* dans un fichier nommé **Nom-Neuf.csv**. Ce document doit contenir les noms de variables mais non les numéros d'individus.

EXERCICE 16 (EXTRAIT DU CONTRÔLE SUR POSTE DE SESSION 1 - 2017-2018)

Vous allez travailler sur les données des employés et services d'une entreprise. Elles sont contenues dans les fichiers `company.csv` et `info_unit.csv` sur Celene. Ces tableaux de données contiennent les informations suivantes.

Dans le fichier `company.csv`, on a les informations suivantes concernant 14999 employés :

- `satisfaction_level` : niveau de satisfaction de l'employé ;
- `last_evaluation` : dernière évaluation ;
- `number_project` : nombre de projets ;
- `average_monthly_hours` : nombre moyen d'heures par mois ;
- `time_spent_company` : temps passé dans l'entreprise ;
- `Work_accident` : s'il a déjà eu un accident de travail ;
- `left` : si l'employé a quitté l'entreprise ;
- `promotion_last_5years` : s'il a eu une promotion au cours des 5 dernières années ;
- `unit` : département ;
- `salary` : niveau de salaire.

Dans le fichier `info_unit.csv`, on a des informations concernant les départements :

- `unit` : département ;
- `effectif_serv` : effectif du département ;
- `nb_accident_serv` : nombre de personnes du département ayant déjà eu des accidents de travail ;
- `nb_left_serv` : nombre de personnes du service ayant quitté l'entreprise.

1. Importer les données du fichier `info_unit.csv` dans un *data frame* intitulé `unit`.
2. Créer un vecteur contenant les taux de départ par département arrondis à 2 chiffres après la virgule.
3. Ajouter ce vecteur au *data frame* `unit` et en supprimer la variable `nb_left_serv`.
4. Importer les données du fichier `company.csv` dans un *data frame* intitulé `comp`.
5. Déterminer le nombre d'employés du département IT ayant déjà eu un accident du travail.
6. Créer un document pdf intitulé `Nom-graph.pdf` comprenant un histogramme des niveaux de satisfaction des employés ayant quitté l'entreprise.
7. Représenter par des diagrammes en boîte la répartition des niveaux de satisfaction selon que l'employé a quitté ou non l'entreprise.
8. Ajouter sur le graphique une droite horizontale colorée au niveau du troisième quartile des niveaux de satisfaction globaux ainsi qu'un texte de la même couleur spécifiant qu'elle correspond au troisième quartile.
9. Créer le tableau de contingence des variables `unit` et `salary`. Ajouter les marges et renommer les titres des colonnes et ligne de marges.
10. Effectuer le tableau des proportions des différents départements arrondis à 3 chiffres après la virgule.
11. Modifier les *levels* de la variable `unit` afin de fusionner les départements dont les effectifs représentent moins de 7% des effectifs de l'entreprise en une modalité `autres`. Puis réaliser un graphique représentant les proportions des différents départements.